The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Chapter

5

The Stack, and Your Shop in One Command

Part II — Standing the Shop Up

Taught in	Week 3
Laboratory	Compose up; verify, break and reset the shop
Assessment	—

This is a class extract. It is *Chapter 5* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Dr. Mustafa Hameed · Department of Information Technology

Generated from `parts/` by `makechapters.py`; do not edit this PDF's source by hand

Chapter 5

The Stack, and Your Shop in One Command

Learning Outcomes

By the end of this chapter you will be able to:

- **Name** the four layers a web shop runs on and **state** what each one is responsible for.
- **Explain** what a container is, and **distinguish** it from a virtual machine and from an installer.
- **Read** a Compose file and **say** what each service, port and volume in it does.
- **Bring up** a working storefront, **verify** it from outside, and **reset** it to a known state.
- **Measure** what the stack costs to start and to store, and **distinguish** the cost paid once from the cost paid daily.

Before You Start

Chapter 4: the decision to use a self-hosted open-source platform, and why. Appendix A must be finished before this chapter — you need a working container runtime, and if it is going to fight you, it will fight you now rather than in Week 9.

Key Terms in This Chapter

stack • LAMP • web server • application server • database server • container • image • volume • port mapping • Docker Compose • service • environment variable • reproducibility • idempotent • pinning • checksum

5.1 Four Layers, and What Each One Does

When you type a shop's address into a browser, four distinct pieces of software answer in turn. Knowing which is which is the difference between fixing a problem and restarting things until it goes away.

Which layer is this problem in?

A diagnostic habit worth building on day one, because it turns "the site is broken" into a question with an answer.

- **Nothing answers at all**, the browser cannot connect: the web server is down, or the port is wrong. Nothing above it has run.
- **A page appears but says "500"**: the web server is fine; the application failed. The reason is in the PHP error log, never on the screen — and Chapter 16 explains why you want it that way.
- **A page appears, looks right, but data is missing or stale**: the application ran and the database answered something unexpected.
- **Everything works and is slow**: usually the database, and Chapter 6 measures exactly that.

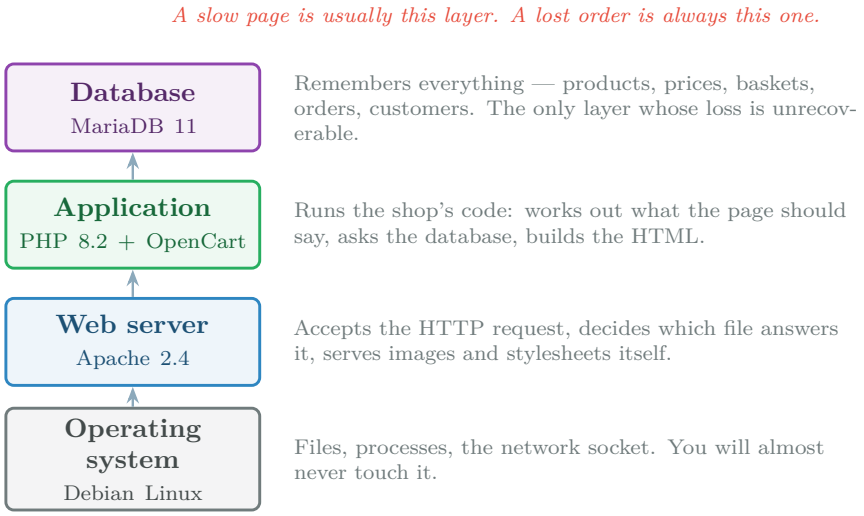


Figure 5.1: The four layers. The traditional name for this arrangement is LAMP — Linux, Apache, MySQL, PHP — and although every letter has alternatives now, the shape has not changed in twenty-five years.

5.2 Containers, and What They Are Not

The traditional way to get those four layers is to install them: a web server, a database, a PHP runtime, each with its own installer, its own configuration file in its own place, and its own way of being slightly different on your machine from on everybody else's.

A **container** replaces that with a package that already contains the software *and* everything it depends on, down to the system libraries, arranged exactly as its author arranged it.

Image, Container, Volume

An **image** is a read-only, layered filesystem containing an application and its dependencies. It is built once and is identical everywhere.

A **container** is a running instance of an image. It has its own filesystem view and its own network identity, but it shares the host's kernel — which is what makes it start in a second rather than a minute.

A **volume** is storage that lives *outside* the container and survives it. This is the one that matters: everything inside a container is thrown away when the container is removed, so anything you want to keep — your database, your shop — must be in a volume.

Three things a container is not

It is not a virtual machine. A VM carries a whole operating system including its own kernel, takes a minute to boot and gigabytes of memory. A container shares the host kernel and starts in about a second. The isolation is correspondingly weaker, which matters in production and not in this course.

It is not a backup. "It is in the container" means it will be gone when the container is replaced, which happens every time you rebuild. Your shop survives because it is in a *volume*, and Chapter 20 is about what happens when somebody has not understood that distinction.

It is not automatically secure. A container with a weak password is a weak password that starts quickly. Chapter 16 attacks this very stack and finds things.

5.3 The Three Services in This Course's Stack

The whole stack is described by one file, `code/docker-compose.yml`, and it declares three **services**.

```
services:
  db:                               # MariaDB 11: remembers everything
    image: mariadb:11
    volumes:
      - dbdata:/var/lib/mysql       # the shop's memory, kept outside
    healthcheck: ...                # "accepts a login", not "started"

  shop:                             # Apache + PHP 8.2 + OpenCart 4.1.0.4
    build: ./shop
    depends_on:
      db: { condition: service_healthy }
    ports:
      - "8090:80"                   # your 8090 -> the container's 80
    volumes:
      - shopdata:/var/www/html

  adminer:                          # a database console in the browser
    image: adminer:latest
    ports:
      - "8091:8080"
```

Three decisions in that file are worth understanding, because each one is a mistake people make.

The healthcheck, not just `depends_on`. Ordering container *starts* is not enough. MariaDB accepts TCP connections several seconds before it will accept a login, so a shop that waits only for the database to have started will try to install itself against a database that refuses it — intermittently, and more often on a fast machine. The condition `service_healthy` waits for the database to say it is actually ready.

The port mapping is a translation, not a setting. `8090:80` means "traffic arriving at port 8090 on my machine goes to port 80 inside the container". The shop always believes it is on port 80. If 8090 is busy on your machine — and on a university laptop it often is — change the left-hand number only.

Named volumes, so that a reset is deliberate. `docker compose down` stops everything and keeps your shop. `docker compose down -v` removes the volumes too, and the next start reinstalls from nothing. That second command is how you recover from a shop you have broken beyond understanding, and it is also how you destroy a semester's work in week 14. It is worth knowing which is which before you need either.

Why the image is built here rather than downloaded

Most teaching material says "pull the OpenCart image". There is no longer one to pull: there has never been an official OpenCart image, and the widely-used third-party one was withdrawn. Every such instruction on the web is now dead.

So `code/shop/Dockerfile` builds it, from the official PHP image plus a *pinned* OpenCart release — pinned by version and by SHA-256 checksum, so that the shop you build this year is the shop a classmate builds next year. Two lines in that file do the real work:

- `ARG OC_VERSION=4.1.0.4` — the exact release.
- `echo "${OC_SHA256} /tmp/oc.zip | sha256sum -c -` — the download really is the file we tested against. A version number alone is not reproducible, because a release asset can be replaced.

There is a second reason, and it is the pedagogical one: Figure 5.1 is a diagram of something, and pulling one opaque image would have left you unable to point at any of it. Here the layers are in the file, in order.

OpenCart ships its own Dockerfile, and it does not work

The release archive contains a `Dockerfile` and a `docker-compose.yml` at the top level. This course uses neither, for two reasons you can verify yourself in five minutes.

The Dockerfile contains a shell syntax error — `docker-php-ext-install zip && && docker-php-ext-enable zip` — so it has never built. The compose file pins `mysql:5.7`, which reached end of life in October 2023 and receives no further security fixes, and starts Redis, Memcached and PostgreSQL that a stock install does not use.

The lesson is not that OpenCart is bad software; it is a mature project that runs a large share of the world's small shops. The lesson is that an artefact being official is evidence and not authority. You are allowed to check, and in this course you are expected to.

5.4 What Happens the First Time

One command brings the whole thing up:

```
docker compose up -d --build
```

On a first run this does rather a lot: downloads about 1.5 GB of images, builds the shop image, starts three containers, waits for the database, and then runs OpenCart's command-line installer, which creates 159 tables and populates them with a demonstration catalogue. On every later run it does almost none of it.

That difference is worth measuring rather than asserting.

Measured: What the Stack Costs to Start

Produced by `code/m05_coldstart.py -scratch`, which removes every image and volume first, on the machine in Appendix A — 4 cores, 7.7 GiB available to the container runtime.

Start	To first 200	What it had to do
From scratch (no images at all)	**338.4 s**	Download 1.5 GB, compile three PHP extensions, fetch and verify OpenCart, install 159 tables. Of this, 214.6 s was the build alone
Cold (images built, no shop)	**34.1 s**	Start three containers, wait for the database, install OpenCart
Warm (the shop already exists)	**16.4 s**	Start three containers. Nothing else

Footprint. Images: shop 882 MB, MariaDB 462 MB, Adminer 164 MB — **1,508 MB** in all. Volumes after installation: database 181 MB, shop files 40 MB. About 1.7 GB of disk for a complete shop.

The ratio is the point. From scratch to warm is **20.6×**. You pay the 338 seconds once, in Week 3, and the 16 seconds perhaps sixty times over the semester. Quoting a single "startup time" for this stack would describe an experience nobody has.

On variance. An earlier run of the same measurement on the same machine gave 17.6 s cold and 7.7 s warm — roughly half — because less was running at the time. Your numbers will differ from these by more than you expect. The *ratio* between cold and warm should not.

5.5 Looking At What You Were Given

The installer leaves you a working shop with a demonstration catalogue. It is worth knowing its shape, because every chapter in Part II changes part of it.

</> The shop, counted

Asked of the database directly, through Adminer or the command line:

```
SELECT (SELECT COUNT(*) FROM oc_product)    AS products,
       (SELECT COUNT(*) FROM oc_category)   AS categories,
       (SELECT COUNT(*) FROM oc_order)      AS orders,
       (SELECT COUNT(*) FROM oc_customer)   AS customers;
```

- **159 tables** in total.
- **19 products** and **38 categories** of demonstration data.
- **0 orders** and **0 customers** — nobody has shopped here yet.
- **18 tables whose name begins `oc_product`.** One product is not one row, and Chapter 6 is about why.

That last number is the one to sit with. A beginner expects a products table. There are eighteen, because a product has descriptions in several languages, images, options, attributes, discounts, related items and category memberships — and each of those is a different relationship.

Common Pitfalls

- **Running `down -v` when you meant `down`.** The `-v` removes the volumes, which is your shop. There is no undo.
- **Editing files inside a running container.** They are gone at the next rebuild. Changes that must survive belong in the image or in a volume.
- **Assuming `up` means ready.** It means *started*. On a first run the shop answers about twenty seconds later; every measurement in this book calls `waitfor.py` first for exactly this reason.
- **Changing the right-hand side of a port mapping.** In `8090:80`, the 80 is the port inside the container and the shop expects it. Change the left number.
- **Treating a container as a backup.** It is the opposite of one.
- **Giving up when it does not start.** Appendix A lists the five failures that account for nearly all of them, and four are a busy port or a runtime that is not actually running.

Try It Yourself: Your shop, up, verified, broken and restored

From here to Chapter 23 this is the same shop. Treat it accordingly.

```
# --- 1. Bring the whole stack up ----------------------------------
# First run: expect about five minutes and 1.5 GB of download.
cd code
cp .env.example .env
docker compose up -d --build

# --- 2. Watch it install ------------------------------------------
# Not optional. You should see "SUCCESS! OpenCart successfully
# installed" go past; if you never see it, nothing below will work.
docker compose logs -f shop

# --- 3. Verify from outside, not from the logs --------------------
# Three URLs must answer 200. waitfor.py checks all three and is the
# answer to "is my shop up?" for the rest of the semester.
python waitfor.py

# --- 4. Look at the shop, as a customer and as the owner ----------
# Storefront   http://localhost:8090/
# Admin        http://localhost:8090/admin/   (admin / admin123)
# Database     http://localhost:8091/         (shop / shoppw / opencart)

# --- 5. Count what you were given ---------------------------------
# In Adminer, or here. 159 tables; 19 products; 0 orders.
docker compose exec db mariadb -ushop -pshoppw opencart \
  -e "SELECT COUNT(*) FROM information_schema.tables \
      WHERE table_schema='opencart';"

# --- 6. Prove the volume is doing what you think ------------------
# Change something in the admin panel first -- edit a product name.
# Then stop WITHOUT -v and start again. Your change is still there.
docker compose down
docker compose up -d
python waitfor.py

# --- 7. Now reset it deliberately ---------------------------------
# -v throws the volumes away. The shop reinstalls from nothing and
# your edit is gone. Do this once, now, while it costs you nothing.
docker compose down -v
docker compose up -d
python waitfor.py

# --- 8. Measure it ------------------------------------------------
# Produces the table quoted above, on YOUR machine.
python m05_coldstart.py --warm
```

What to notice. Step 6 and step 7 differ by two characters and by everything else. Having done both deliberately, in a week when the shop contains nothing you care about, is the cheapest possible way to learn the difference.

Also notice that step 3 does not trust step 2. The log said the install succeeded; step 3 asks the shop. Throughout this course, the thing that reports success and the thing that is true are kept apart on purpose.

Checkpoint — Test Yourself

1. A classmate's shop returns "500 Internal Server Error" on every page. Which of the four layers in Figure 5.1 are known to be working, which is known to have failed, and where would you look first?

2. Explain why `depends_on` alone is not enough to order the database before the shop, and describe the failure it produces.
3. You run `docker compose down -v` by mistake in Week 12. State exactly what is lost, what is not, and what would have had to exist beforehand for this to be a minor inconvenience.

Chapter Summary

- A shop runs on four layers — operating system, web server, application, database. Knowing which layer a symptom belongs to is most of diagnosis; only the database's loss is unrecoverable.
- An image is a built filesystem, a container is a running instance of one, and a volume is the only part that survives the container. A container is not a virtual machine, not a backup, and not secure by itself.
- The whole stack is three services in one Compose file: database, shop, database console.
- `depends_on` orders starts, not readiness. A healthcheck is what makes the shop wait until the database will actually accept a login.
- `down` keeps your shop; `down -v` destroys it. Learn the difference in Week 3.
- The image is built from a pinned release verified by checksum, because no maintained OpenCart image exists to pull and because a version number alone is not reproducible.
- Measured on this machine: 338.4 s from scratch, 34.1 s cold, 16.4 s warm — a ratio of 20.6× between the cost paid once and the cost paid daily. The stack occupies about 1.7 GB.
- The installed shop has 159 tables and 19 products, and eighteen of those tables are about products. One product is not one row.

Review Questions

1. (MCQ) In the port mapping `8090:80`, the 80 is: (a) the port on your machine; (b) the port inside the container; (c) the database port; (d) an arbitrary label.

2. (MCQ) Which survives `docker compose down`? (a) Nothing; (b) the containers; (c) data written to a named volume; (d) files edited inside a running container.

3. (Short) Distinguish a container from a virtual machine, and give one consequence of the difference that matters in this course.

4. (Short) Why does this course build the shop image from a pinned release rather than pulling a ready-made one? Give both reasons.

5. (Short) A page loads but shows yesterday's price. Which layer is at fault, and what are the two most likely explanations?

6. (Applied) Your stack will not start: port 8090 is already in use. Describe exactly what you would change, in which file, and what you would then have to change in the shop's own configuration for links to keep working. (Look at `OC_HTTP_SERVER` before answering.)

7. (Applied) Design a measurement that would show whether the *database* or the *application* layer is responsible for a slow category page, using only tools in this chapter. State what you would record, at what catalogue sizes, and what result would point at each layer. Chapter 6 runs your experiment; write it first.